Java Interview Bible — Core + Advanced (Printable)

Friendly, detailed interview notes (not one-liners). Includes fast "1-minute answer" + deeper explanation + code where useful.

Updated: 2026-02-25

Covers: Core Java • Collections • Concurrency • JVM/GC • Java 8+ • JDBC • Servlets basics • Design

Best for: 2–6 years experience interviews

Table of Contents

> **How to use:** For each question, learn the **1-minute answer** first, then read the **deep answer**. Practice speaking it.

---

1) Java Basics

Q1. What is Java and why is it platform independent?

1-minute answer

- Java compiles to **bytecode** (.class), not machine code.
- Bytecode runs on **JVM** available on each OS.
- So the same .class runs on Windows/Linux/Mac.

Deep answer

javac converts source into bytecode. Then JVM interprets/JIT-compiles bytecode into native instructions at runtime. Because the JVM is OS-specific but bytecode is universal, Java achieves "write once, run anywhere".

Very common | Core fundamentals

Q2. JDK vs JRE vs JVM

1-minute answer

- **JVM**: runs bytecode
- **JRE**: JVM + standard libraries to run apps
- **JDK**: JRE + dev tools (javac, debugger)

Deep answer

JDK is for development; JRE is for running. JVM is the engine doing class loading, execution, GC, and JIT compilation.

Q3. Why is main method `public static void main(String[] args)`?

1-minute answer

- **public**: JVM must call it
- **static**: no object required
- **void**: no return expected
- **String[]**: command-line arguments

Deep answer

JVM looks for an entry method with a fixed signature. You can overload `main`, but JVM starts with this exact one.

2) OOP (Core)

Q4. Explain Encapsulation, Inheritance, Polymorphism, Abstraction

1-minute answer

- **Encapsulation**: hide data using private, expose via methods
- **Inheritance**: reuse behavior using `extends`
- **Polymorphism**: same interface, different behavior (overload/override)
- **Abstraction**: expose essential behavior, hide implementation

Deep answer

Encapsulation reduces accidental misuse. Inheritance enables reuse but can tightly couple. Polymorphism enables substitutability (parent reference, child object). Abstraction reduces complexity by hiding details.

Q5. Abstract class vs Interface (when to use which?)

1-minute answer

- Use **interface** for "capability/contract" and multiple inheritance of type.
- Use **abstract class** when you need shared state + shared base code.

Deep answer

Abstract classes can have fields and constructors. Interfaces are best for designing clean APIs. Since Java 8, interfaces can have `default` methods but still avoid shared mutable state.

Q6. Overloading vs Overriding

1-minute answer

- **Overloading**: same name, different params, compile-time decision
- **Overriding**: child redefines parent method, runtime decision

Deep answer

Overriding works with dynamic dispatch. Overloading is resolved using reference types and method signatures at compile time.

Q7. Why static methods are not overridden?

1-minute answer

- Static belongs to **class**, not object
- So it's **method hiding**, not overriding

Deep answer + example

```
class A { static void m(){ System.out.println("A"); } }
class B extends A { static void m(){ System.out.println("B"); } }

A obj = new B();
obj.m(); // A (because static resolved by reference type A)
```

---

3) String + String Pool

Q8. Why String is immutable?

1-minute answer

- Security + thread-safety
- String pool memory optimization
- Stable hashCode for HashMap keys

Deep answer

When you "modify" a string, Java creates a new object. This makes sharing safe and enables pooling.

Q9. == vs equals() for Strings

1-minute answer

- == compares **reference**
- equals compares **content**

Deep answer + example

```
String a = "java";
String b = "java";
System.out.println(a == b);      // true (same pooled literal)
System.out.println(a.equals(b)); // true

String c = new String("java");
System.out.println(a == c);      // false (different object)
System.out.println(a.equals(c)); // true
```

Q10. String vs StringBuilder vs StringBuffer

1-minute answer

- **String**: immutable
- **StringBuilder**: mutable, fast, not thread-safe
- **StringBuffer**: mutable, thread-safe, slower

Deep answer

For repeated concatenation in loops, use StringBuilder to avoid creating many intermediate String objects.

---

4) Wrapper / Boxing

Q11. What is boxing/unboxing and what is a common pitfall?

1-minute answer

- Boxing: primitive → wrapper
- Unboxing: wrapper → primitive
- Pitfall: **NPE** when unboxing null

Deep answer + example

```
Integer x = null;
// int y = x; // NullPointerException due to unboxing
```

Q12. Why `Integer 100` compares true with == but 200 is false?

1-minute answer

- Because of **Integer cache** (typically -128 to 127)
- Use `equals()` for value comparison

Deep answer + example

```
Integer a = 100, b = 100;
System.out.println(a == b); // true (cached)

Integer c = 200, d = 200;
System.out.println(c == d); // false (new objects)

System.out.println(c.equals(d)); // true
```

---

5) Exception Handling

Q13. Checked vs Unchecked exceptions

1-minute answer

- **Checked**: must handle/declare (IOException)
- **Unchecked**: runtime (NPE, ArithmeticException)

Deep answer

Checked exceptions are for recoverable conditions enforced by compiler. Unchecked exceptions often indicate programming errors or invalid input states.

Q14. `throw` vs `throws`

1-minute answer

- `throw`: actually throws exception
- `throws`: declares method may throw

Deep answer + example

```
static void validateAge(int age) throws IllegalArgumentException {
  if (age < 18) throw new IllegalArgumentException("Not eligible");
}
```

Q15. try-with-resources (preferred)

1-minute answer

- Auto-closes resources implementing `AutoCloseable`
- Avoids resource leaks

Example

```
try (java.io.FileReader fr = new java.io.FileReader("file.txt")) {
  // use fr
} // auto close
```

6) Collections Framework (Internals)

Q16. List vs Set vs Map

1-minute answer

- **List**: ordered, duplicates allowed
- **Set**: no duplicates
- **Map**: key-value; keys unique

Deep answer

Choose based on requirement: duplicates? ordering? key lookup? Example: username → user object is naturally a Map.

Q17. ArrayList vs LinkedList (when to use)

1-minute answer

- ArrayList: fast random access
- LinkedList: faster insert/delete in middle (if you already have node)

Deep answer

ArrayList is backed by array → resizing copies elements. LinkedList stores nodes → extra memory + slow index access.

Q18. HashMap internal working (most asked)

1-minute answer

- HashMap uses `hashCode()` to pick bucket
- Uses `equals()` to match key in bucket
- Collisions handled using list/tree

Deep answer (interview-ready)

Steps: compute hash → index → insert/lookup. If collision happens, multiple entries go in same bucket. When collisions exceed threshold, bucket may become a tree for faster lookup. Resizing rehashes entries into a bigger table.

Important: if a key's fields change after insert, hashCode changes → map may not find it again.

Q19. HashMap vs Hashtable vs ConcurrentHashMap

1-minute answer

- Hashtable: synchronized, legacy
- HashMap: not thread-safe
- ConcurrentHashMap: thread-safe with better concurrency

Deep answer

ConcurrentHashMap avoids locking the whole map for reads/writes in many cases, improving throughput in multi-threaded systems.

Q20. Comparable vs Comparator

1-minute answer

- Comparable: natural order inside class (`compareTo`)
- Comparator: custom order outside class (`compare`)

Example

```
// Comparator example: sort by salary
list.sort((e1, e2) -> Integer.compare(e1.salary, e2.salary));
```

7) Multithreading + Concurrency

Q21. Process vs Thread

1-minute answer

- Process: independent memory space
- Thread: shares memory inside same process

Deep answer

Threads are lighter but need synchronization because shared memory can cause race conditions.

Q22. Ways to create a thread + why Runnable is preferred

1-minute answer

- Extend Thread
- Implement Runnable (preferred)
- Callable + ExecutorService

Deep answer

Runnable separates “task” from “thread”. It also allows your class to extend another class (since Java has single inheritance).

```
Runnable task = () -> System.out.println("Running in thread: " + Thread.currentThread().getName());
new Thread(task).start();
```

Q23. `synchronized`: object lock vs class lock

1-minute answer

- `synchronized(obj)` → locks that object
- `synchronized(ClassName.class)` → locks class-level (static lock)

Deep answer + example (Singleton style)

```
public class Singleton {
  private static volatile Singleton instance;

  private Singleton() {}

  public static Singleton getInstance() {
    if (instance == null) { // first check
      synchronized (Singleton.class) { // class lock (one for all threads)
        if (instance == null) { // second check
          instance = new Singleton();
        }
      }
    }
    return instance;
  }
}
```

Why class lock? Because method is static and must guard a single shared instance across all callers.

Q24. wait/notify vs sleep

1-minute answer

- `sleep()`: pauses, does NOT release lock
- `wait()`: releases lock and waits for notify

Deep answer

`wait/notify` is for inter-thread communication inside synchronized blocks. Always call `wait()` in a loop checking condition.

Q25. volatile vs Atomic vs synchronized

1-minute answer

- **volatile**: visibility guarantee, not atomic for ++
- **Atomic**: atomic ops via CAS
- **synchronized**: mutual exclusion + visibility

Deep answer

Use Atomic for counters. Use synchronized/locks when you need multiple steps as one atomic critical section.

---

8) JVM Internals + GC

Q26. JVM memory areas (high-level)

1-minute answer

- **Heap**: objects
- **Stack**: per-thread frames/local vars
- **Metaspace**: class metadata

Deep answer

Heap is shared by threads. Stack is private per thread and manages method calls. Metaspace stores class information loaded by classloaders.

Q27. Garbage Collection: generational idea + why "most objects die young" matters

1-minute answer

- Young gen cleaned frequently (fast)
- Old gen cleaned less frequently
- Optimized because most objects are short-lived

Deep answer

Generational GC reduces cost by focusing frequent collections on the young generation where garbage is usually highest.

Q28. How memory leaks happen in Java

1-minute answer

- GC can't collect objects that are still referenced
- Common causes: static caches, listeners not removed

Deep answer

Leak = unwanted long-lived references. Fix by removing references, using weak references, bounded caches, and proper lifecycle cleanup.

---

9) Java 8+ (Lambda / Streams / Optional)

Q29. Functional Interface + why important

1-minute answer

- Exactly one abstract method
- Used by lambdas and method references

Deep answer

Compiler can match a lambda to a functional interface. Examples: Runnable, Comparator, Supplier, Function.

Q30. Stream API: map vs flatMap

1-minute answer

- map: 1 element → 1 element
- flatMap: 1 element → many → flattened

Example

```
// flatMap example
List<List<Integer>> lists = List.of(List.of(1,2), List.of(3,4));
List<Integer> all = lists.stream()
  .flatMap(List::stream)
  .toList(); // [1,2,3,4]
```

Q31. Optional: how to use properly

1-minute answer

- Use as return type to express “may be absent”
- Avoid using Optional for fields everywhere

Deep answer

Prefer `orElseGet` when default computation is expensive. Avoid `get()` without check.

---

10) JDBC

Q32. JDBC flow (steps)

1-minute answer

- Get Connection
- Create PreparedStatement
- Execute
- Read ResultSet
- Close resources

Deep answer + safe example

```
String sql = "SELECT id, name FROM users WHERE email = ?";
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {

  ps.setString(1, email);
  try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
      int id = rs.getInt("id");
      String name = rs.getString("name");
    }
  }
}
```

Q33. Statement vs PreparedStatement

1-minute answer

- PreparedStatement prevents SQL injection
- Better for repeated queries

Deep answer

PreparedStatement uses parameter binding. It avoids string concatenation bugs and improves security.

Q34. Transactions: commit/rollback

1-minute answer

- Disable auto-commit
- Commit if all steps succeed
- Rollback on error

Example

```
con.setAutoCommit(false);
try {
  // step1
  // step2
  con.commit();
} catch (Exception e) {
  con.rollback();
  throw e;
} finally {
  con.setAutoCommit(true);
}
```

---

11) Servlets / Web Basics

Q35. What is a Servlet and its lifecycle?

1-minute answer

- Servlet handles HTTP request/response
- Lifecycle: init → service(doGet/doPost) → destroy

Deep answer

Servlet container (Tomcat) manages servlet objects and thread-per-request model (commonly). Thread safety matters.

Q36. GET vs POST

1-minute answer

- GET: read/fetch, params in URL
- POST: create/submit, data in body

Deep answer

GET is cacheable and should be idempotent. POST is used for operations that change server state.

---

12) Design + Best Practices

Q37. SOLID principles (what and why)

1-minute answer

- S: one reason to change
- O: extend without modifying
- L: child should be substitutable
- I: small focused interfaces
- D: depend on abstractions

Deep answer

SOLID reduces coupling and improves maintainability. In interviews, give one real example (e.g., strategy pattern for OCP).

Q38. Why composition often better than inheritance?

1-minute answer

- Less coupling, easier testing
- Avoids fragile base class issues

Deep answer

Inheritance exposes parent internals to child. Composition lets you change behavior by swapping components (strategy/decorator).

Q39. Singleton pitfalls + best approach

1-minute answer

- Thread safety, reflection, serialization can break it
- Prefer enum singleton in many cases

Deep answer

Double-check locking needs `volatile`. Enum singleton is simplest and safest against serialization issues.

---

13) Tricky & Frequently Asked

Q40. Why equals() and hashCode() must be consistent?

1-minute answer

- HashMap finds bucket using hashCode
- Then matches key using equals

Deep answer

If two equal objects return different hashes, they may go into different buckets → lookup fails or duplicates appear in sets.

Q41. Can we overload main method?

1-minute answer

- Yes, but JVM starts with exact signature main(String[])

Example

```
public static void main(String[] args) { main(1); }
public static void main(int x) { System.out.println("overloaded"); }
```

---

14) 1-day / 4-hour Quick Prep Plan

4-hour plan (highest ROI)

1. **45 min:** OOP + abstract/interface + overriding/overloading + static pitfalls
2. **60 min:** Collections + HashMap internals + equals/hashCode + Comparable/Comparator
3. **45 min:** Exception handling + custom exception + try-with-resources
4. **45 min:** Threads: synchronized, locks, wait/notify, volatile vs atomic
5. **45 min:** Java 8: lambda, functional interface, stream patterns, optional

---

Tip: Practice speaking 8–10 answers out loud. Interview = communication + clarity.

What to add next (optional modules)

- JVM/GC deeper (GC types, tuning basics)
- JDBC + transactions (already included above)
- Servlet thread safety + sessions/cookies
- Basic REST and HTTP status codes (if backend roles)

End of notes.